САМОСТОЯТЕЛЬНАЯ РАБОТА СТУДЕНТА

Вариант 14
Журнал событий веб-сервиса

Технология: Apache Cassandra | Time-bucketing & Hot-partition control

Дисциплина: Проектирование и применение NoSQL-технологий
3 курс, ОП «Информационные системы»

Проблема и цель

Проблема

Микросервисы генерируют поток логов.
Без правильной модели данных:
• unbounded partitions
• hot partition на популярном сервисе
• медленные «последние N»
• рост latency при увеличении объёма

Цель

Реализовать прототип журнала событий на Cassandra с:

• daily / hourly bucketing
• эффективным «последние N»
• фильтром по level
• демонстрацией anti-hot-partition
• ≥ 30 синтетических записей
• полным CRUD и тестами

Требования

Функциональные (≥ 6)

- FR1 Регистрация сервисов
- FR2 Запись события (CREATE)
- FR3 Последние N за день
- FR4 Фильтр по level
- FR5 Диапазон [t1,t2] (кросс-день)
- FR6 Удаление события
- FR7 COUNT в партиции
- FR8 Hourly bucket (HV)

Нефункциональные (≥ 3)

- NFR1 Bounded partitions (день/час)
- NFR2 Предсказуемая latency LIMIT
- NFR3 TTL 30 / 7 дней
- NFR4 Горизонтальное масштабирование

Access Patterns: AP1–AP8 (см. отчёт / README)

Выбор технологии — ADR

Альтернатива	Плюсы	Минусы для задачи
MongoDB	Гибкая схема	Слабый контроль размера партиций
Redis	Скорость	RAM, не для long-term логов
PostgreSQL	SQL, mature	Вертикальное масштабирование
Elasticsearch	Full-text	Избыточен, дорого
Cassandra	**Write-scale, TWCS, buckets**	**Нет JOIN (решается denorm)**

Модель данных

```
PRIMARY KEY ((service_name, day), event_time, event_id)
CLUSTERING ORDER BY (event_time DESC, event_id DESC)
```

events_by_service_day

Основной журнал
PK: (svc, day)
TTL 30 дней

events_by_service_hour

Anti-hot-partition
PK: (svc, hour)
TTL 7 дней

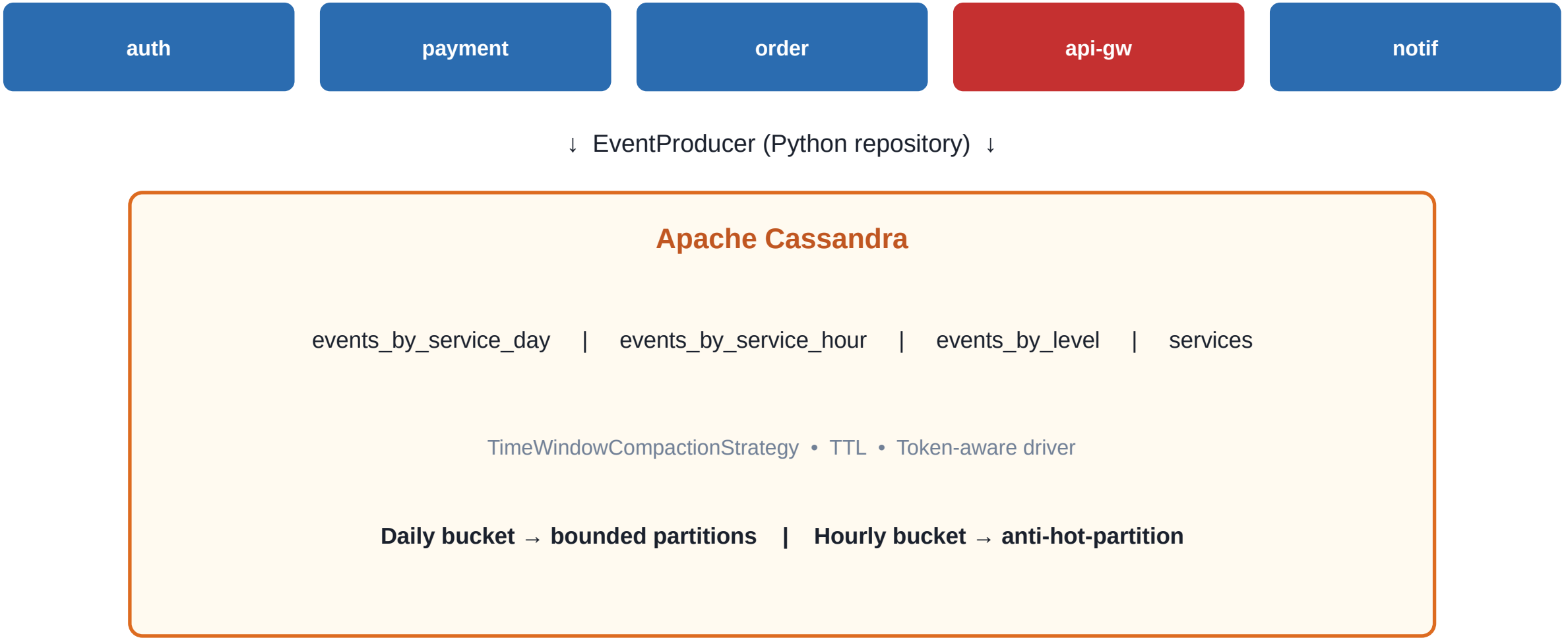
events_by_level

Денормализация
PK: (svc, day, level)
Быстрый фильтр

services

Метаданные
PK: service_name

Архитектура



Реализация

Стек: Python 3 + cassandra-driver + pytest + Docker Cassandra

`src/schema.py`	CQL DDL: keyspace, 4 таблицы, TWCS, TTL
`src/models.py`	Event, day_bucket(), hour_bucket()
`src/repository.py`	CRUD + AP1–AP8 (prepared statements)
`scripts/seed.py`	≥ 60 синтетических событий за 3 дня
`scripts/demo_queries.py`	≥ 8 запросов, 2 нетривиальных (Q5, Q8)
`tests/`	≥ 3 сценария: create/read, level, delete+count

Результаты запросов и тестов

Демо-запросы (фрагмент)

- Q2 Latest 5 — auth-service @ today
- Q3 ERROR — payment-service
- Q5 [non-trivial] order-service 48h cross-day
- Q6 Hourly bucket — api-gateway
- Q8 [non-trivial] Partition size control

Тесты (pytest)

✓ create + read_latest (newest-first order)

✓ read_by_level (ERROR vs INFO)

✓ delete + count (row disappears)

✓ time_range cross-day (bonus)

Ограничения, выводы, GitHub

Ограничения прототипа

• Один узел, RF=1 • Кросс-дневные запросы последовательные • Нет app-level auth • Синтетика

Выводы

Реализован полный прототип, удовлетворяющий всем требованиям варианта 14 и критериям СРО:
≥ 6 FR + ≥ 3 NFR, ≥ 8 Access Patterns, ADR, CRUD, ≥ 60 записей, ≥ 8 запросов (2 non-trivial), ≥ 3 теста, README, отчёт, презентация. Daily/hourly bucketing демонстрирует контроль партиций и anti-hot-partition.

GitHub-репозиторий
(ссылка размещается в README после публикации)